CMSC 202 Spring 2026

Project 1 – Word Search

Assignment: Project 1 – Word Search

Due Date: Tuesday, February 24th at 8:59pm on GL

Value: 80 points

1. Overview

In this project, you will:

- Write a program that calls multiple functions,
- Use simple file input/output, and
- Pass arrays to functions.

2. Background

A word search is a classic puzzle where you scan a grid of letters to find a hidden list of words. The words can appear horizontally, vertically, diagonally, and sometimes even backwards, which turns a simple-looking grid into a fun pattern-finding challenge. Most word searches come with a theme such as animals, holidays, or science terms. This makes it feel like solving them is a bit like “hunting” for related vocabulary. They’re popular in classrooms and casual puzzle books because they’re easy to start, satisfying to finish, and they quietly build skills like spelling, attention to detail, and visual scanning.

This project is designed to help students practice programming concepts such as loops, constants, arrays, and file input in C++.

Here is an image of what our base project will look like. All of the words will be left-to-right only.

Figure 1. Sample of Left-to-Right Words

3. Assignment Description

Your assignment is to develop the code in **`proj1.cpp`** to read in an input file that has a 10 x 10 grid of letters and a list of words (always exactly 6). It provides a great introduction to fundamental programming concepts such as arrays, functions, string manipulation, user input handling, and basic program logic. The base project will only have words that go from left to right. There will be no diagonal, backwards, or up and down words in the base project. There is an available extra credit assignment at the end of the project document that can only be completed when you have successfully completed the base project.

4. Starting Files

To copy the starting files to your working directory, follow these instructions:

1. Navigate to your project 1 directory (Recommend that you use **`202/projects/proj1`** or something similar).
2. Run this copy command:

   `cp /afs/umbc.edu/users/j/d/jdixon/pub/cs202/proj1/* .`
3. Check that the copy worked:

   `ls`

and you should see the following:
`proj1_sample.txt, proj1_sample_extra.txt proj1_1.txt, proj1_2.txt, and proj1_extra.txt.`

The base project (80pts) uses these files:

`proj1_sample.txt (sample run on the base project),`

`proj1_1.txt (sample input file for base project), and`

`proj1_2.txt (sample input file for base project).`

The *optional* extra credit uses (up to 8pts) uses these files:

`proj1_sample_extra.txt (sample run of the extra credit), and proj1_extra.txt (sample input file for extra credit).`

5. Requirements:

This is a list of the requirements of this application. For this project, it is up to you exactly how you want to implement it. For you to earn all the points, however, you will need to meet all the defined requirements.

- You must follow the coding standard as defined in the CMSC 202 coding standards (found on Blackboard under course materials). This includes comments as required.
- The project must be turned in on time by the deadline listed above on GL. The project must be able to run on GL.

- This assignment is completely individual. You may not work with anyone else on this project. Do not use ChatGPT or any other generative AI. Do not share your code or project requirements with anyone who doesn't work for CMSC 202 at UMBC.
- The project must be completed in C++. You may not use any libraries or structures that we have not learned in class. These are the only libraries that you are allowed to use in this project **`<iostream>`**, **`<ctime>`**, **`<string>`**, **`<fstream>`**, **`<iomanip>`**, **`<cmath>`**, **and** **`<cstdlib>`**. You should only use **`namespace std`**.
- All user input must be validated. However, the user is expected to always enter the correct data type. i.e., if the user is asked to enter an integer, they will. If they are asked to enter a character, they will. You do not need to worry about checking for correct data types.
- Specific coding requirements include:
    - You are creating exactly one file, **`proj1.cpp`** to turn in. All functions including main should be in that file.
    - The project must run as closely to the sample output here and the provided sample output file (**`proj1_sample.txt`**) as possible. Significant deviation from the sample output may result in a reduction in points.
    - Must use at least 5 different *meaningful* functions. (**`main`** + 5 additional functions)
    - Must pass at least two arrays to a function. At least one must be multidimensional.
    - Must have at least one nested loop.
    - Must use at least one do..while loop.
    - Must use at least one constant.
    - Must be able to read in different word search input files. The user will be prompted for the name of the input file. Files can be named anything. Input files will always have a 10x10 character grid and then exactly 6 words below it.

- Must use input validation (assume the data is the correct type).
- No explicit pass-by-reference. No pointers. No vectors. No structs, classes, or objects. No recursion. No EOF usage. No global variables. No for-each loops. No auto. No warnings or errors when submitted.
- You cannot use any structures, functions, or libraries that we have not already explicitly covered at the time of this project being assigned.
- No ChatGPT or other AI based code generation is allowed.
- No warnings or errors when you turn it in.

6. Recommendations

You are free to implement this with your own functions. To complete this word-search project successfully, we recommend approaching it as a sequence of small, testable steps rather than trying to build the whole game at once. Start by confirming you can read the input file correctly: load the 10×10 grid and the list of 6 words, then immediately print both to verify the file format and your indexing match what you expect. Next, implement the “core” feature—given a start coordinate and an end coordinate, build the word by walking through the grid. Treat this as the engine of the entire program: once it works reliably, checking the word list and building the game loop becomes straightforward. Remember, the base project only needs to be able to build words from left to right.

Before building the word, validate user input carefully: coordinates must be within bounds, and the start/end pair will always be in the same row; if the guess is invalid, print a helpful message and reprompt rather than crashing or producing nonsense. After that, add the “game” layer: loop until all six words are found, and track progress so a word can only be counted once (for example, by marking found words or storing a separate boolean list).

Throughout development, test with small, known cases—pick a word you know is in the grid and try its coordinates first—then expand to trickier directions and backwards guesses. Finally, keep your program user-friendly: show the grid with coordinate labels, display the remaining words each round,

and clearly distinguish between if they successfully found a word on the list or the coordinates were invalid, or they didn't find a word on the list.

7. Sample Input and Output

In the sample output below, user input is colored **blue** for clarity. After compiling and running proj1, here is a partial sample run:

```
[jdixon@linux1 proj1]$ ./proj1
What is the file name?
proj1_1.txt
   0 1 2 3 4 5 6 7 8 9
 0 U M B C Q R T Y P L
 1 K J H G F D S A Z X
 2 M N B V C S T A C K
 3 W V C L A S S K E T
 4 P O I N T E R L M Q
 5 Z P X N V Q W E R T
 6 A S D F G U H J K O
 7 P O K E F Y U I O P
 8 Z X C V B Q U E U E
 9 D E B U G P L M N T
UMBC
STACK
CLASS
POINTER
QUEUE
DEBUG
What are the coordinates of the start of the word? (row space
column)
9 0
What are the coordinates of the end of the word? (row space column)
```

```
9 4
Hooray! You found the word, "DEBUG" !
   0 1 2 3 4 5 6 7 8 9
 0 U M B C Q R T Y P L
 1 K J H G F D S A Z X
 2 M N B V C S T A C K
 3 W V C L A S S K E T
 4 P O I N T E R L M Q
 5 Z P X N V Q W E R T
 6 A S D F G U H J K O
 7 P O K E F Y U I O P
 8 Z X C V B Q U E U E
 9 D E B U G P L M N T
UMBC
STACK
CLASS
POINTER
QUEUE
What are the coordinates of the start of the word? (row space
column)
0 10
What are the coordinates of the end of the word? (row space column)
0 0
INVALID COORDINATES
What are the coordinates of the start of the word? (row space
column)
2 5
What are the coordinates of the end of the word? (row space column)
2
9
Hooray! You found the word, "STACK" !
   0 1 2 3 4 5 6 7 8 9
```

```
 0 U M B C Q R T Y P L
 1 K J H G F D S A Z X
 2 M N B V C S T A C K
 3 W V C L A S S K E T
 4 P O I N T E R L M Q
 5 Z P X N V Q W E R T
 6 A S D F G U H J K O
 7 P O K E F Y U I O P
 8 Z X C V B Q U E U E
 9 D E B U G P L M N T
UMBC
CLASS
POINTER
QUEUE
What are the coordinates of the start of the word? (row space
column)
7 0
What are the coordinates of the end of the word? (row space column)
7 3
Sorry, that isn't one of the words. Please try again.
   0 1 2 3 4 5 6 7 8 9
 0 U M B C Q R T Y P L
 1 K J H G F D S A Z X
 2 M N B V C S T A C K
 3 W V C L A S S K E T
 4 P O I N T E R L M Q
 5 Z P X N V Q W E R T
 6 A S D F G U H J K O
 7 P O K E F Y U I O P
 8 Z X C V B Q U E U E
 9 D E B U G P L M N T
```

```
UMBC
CLASS
POINTER
QUEUE
What are the coordinates of the start of the word? (row space
column)
```

See the provided **`proj1_sample.txt`** file for additional examples.

8. Compiling and Running

To compile your program, enter the following command at the Linux prompt:

`g++ -Wall proj1.cpp -o proj1` (use this command to show warnings – which should be eliminated before turning your code in)

This command runs the GNU C++ compiler (**`g++`**). The option **`-Wall`** instructs the compiler to be verbose in its production of warning messages; the option **`-o proj1`** (hyphen followed by the letter "o", not the digit zero), instructs the compiler to give the executable program the name **`proj1`**. If the program compiles correctly, the executable file **`proj1`** will be created in the current directory. *Your project files should have no warnings or errors when turned in.*

At the Linux prompt, enter the command **`./proj1`** to run your program. It should look like the sample output provided above.

9. Completing your Project

When you have completed your project, you can copy it into the submission folder. You can copy your files into the submission folder as many times as you like (before the due date). We will only grade what is in your submission folder.

Test your code!

For this project, you should submit these files to the **`proj1`** subdirectory:

`proj1.cpp` — should include your implementations of the required functions.

After you have set up your symbolic link (as from lab 1), copy the project files into your proj1 folder. Execute these commands:

a. cd to your project 1 folder. An example might be:

```
cd ~/202/projects/proj1
```

b. `cp proj1.cpp ~/cs202proj/proj1`

You can check to make sure that your files were successfully copied over to the submission directory by entering the command:

```
ls ~/cs202proj/proj1
```

You can check that your program compiles and runs in the `proj1` directory, but please clean up any `.o` and executable files. Again, do not develop your code in this directory and you should not have the only copy of your program here. Only submit the `proj1.cpp` file.

IMPORTANT: If you want to submit the project late (after the due date), you will need to copy your files to the appropriate late folder. If you can no longer copy the files into the proj1 folder, it is because the due date has passed. You should be able to see your proj1 files, but you can no longer edit or copy the files in to your proj1 folder. (They will be read only)

- If it is 0-24 hours late, copy your files to `~/cs202proj/proj1-late1`
- If it is 24-48 hours late, copy your files to `~/cs202proj/proj1-late2`
- If it is after 48 hours late, it is too late to be submitted.

10. (Optional) Extra Credit

If you have completed the base project and you have additional time, you can **optionally** complete this extra part to earn a maximum of 8 additional points. You can view the sample output from this extra credit in the `proj1_sample_extra.txt` file. We will not be able to help debug or answer any questions related to the extra credit (other than clarifying

requirements). There is a provided test file for the extra credit called, **`proj1_extra.txt`**. This file has words in different directions.

****STOP**** - Turn in your base project first. <u>You must turn in two separate **`proj1.cpp`** files</u>.

1. Base project: Turn in normally (80pts)
2. Extra Credit: Create a new directory in **your working directory** named **`extra_credit`** and copy the **`proj1.cpp`** file into the extra credit directory to edit. When you are finished updating the **`files`** with the new requirements, you are just going to copy the required files to a second submission directory (up to 8pts).

For the base implementation of this project, you are only searching for words in grid from left to right (They will always be in the same row). For the extra credit, you will be making it so that words can be formed in any of the 8 base directions.

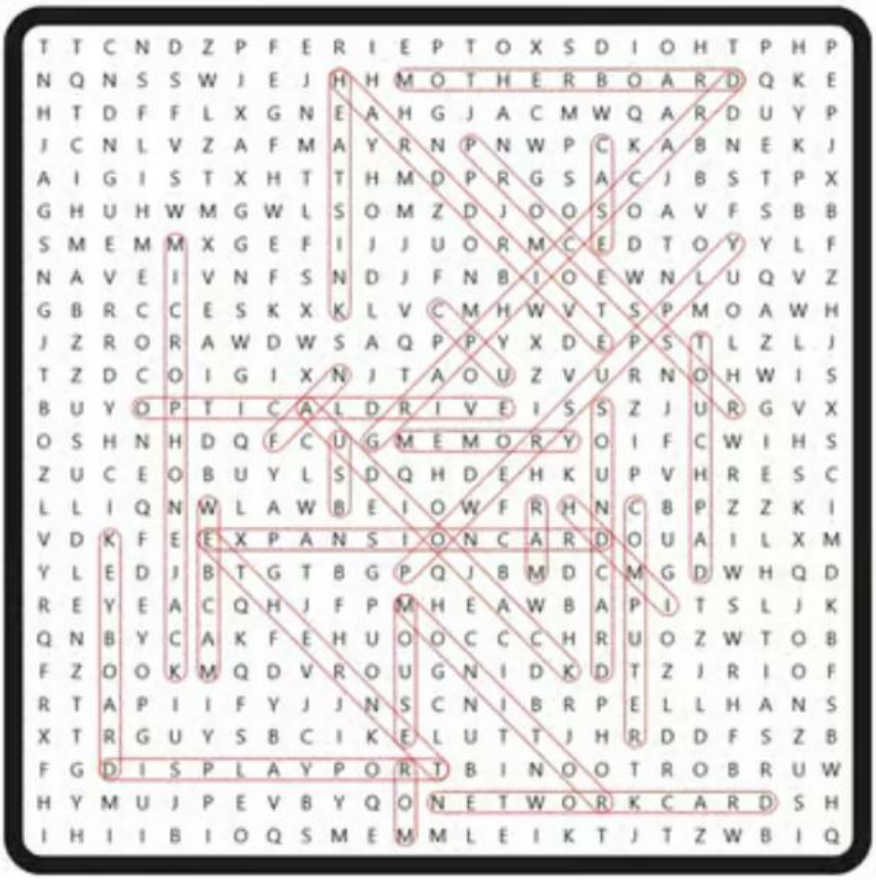

Figure 2. Words in any Direction (extra credit only)

To be clear, the assigned version of this project is worth 80 points, the extra credit is only worth 8. You must complete the assigned version before attempting to earn the extra credit. If you do not complete the assigned version, we will not grade the extra credit. You may not turn in the extra credit late.

To complete the extra credit version of project 2, create a new folder in your project 2 directory named **`extra_credit`**. Copy your base project files into the **`extra_credit`** directory.

```
cp * extra_credit
```

Notice there is a space between the * and the **`extra_credit`**.

To turn in the extra credit, create an **`extra_credit`** folder in your submission folder:

```
mkdir ~/cs202proj/proj2/extra_credit
```

Then submit your **`extra_credit`** files using this command:

```
cp proj1.cpp  ~/cs202proj/proj1/extra_credit
```

Final Check:

MAKE SURE THAT YOU HAVE SUBMITTED TWO VERSIONS: BASE VERSION in

```
ls ~/cs202proj/proj1
```

AND THE EXTRA CREDIT in

```
ls ~/cs202proj/proj1/extra_credit
```

You MUST turn in two completely separate versions (base version and extra credit version) of all files to be eligible for the extra credit.